CPU scheduling - lec 7

prof said this is imp for the exam!! goal = keep the cpu busy and keep response time low.
the scheduler picks the next process from the ready queue. the dispatcher does the actual switch (context switch).
context switch time is pure overhead, no useful work gets done. maybe ~10 microsec but it adds up if the quantum is tiny.

FCFS

first come first served. non-preemptive. just a simple queue.
problem = convoy effect: one long job at the front makes all the short ones wait behind it.
eg bursts 24, 3, 3 (all arrive at 0) -> waiting times 0, 24, 27 so avg wait = 17 ms.

SJF

shortest job first. gives the minimum avg waiting time (provably optimal) but we cant know the next burst length in advance.
so we predict it with exponential averaging: tau_next = alpha * t_n + (1 - alpha) * tau_n, alpha is usually 0.5.
the preemptive version is called SRTF (shortest remaining time first).
starvation: long jobs may never run if short ones keep coming.

Round robin

each process gets a time quantum q, then goes to the back of the queue. preemptive. good for time sharing.
if q too large -> becomes FCFS. if q too small -> too many context switches.
rule of thumb from lecture: about 80% of the cpu bursts should be shorter than q.
eg q = 4 ms with bursts 24, 3, 3: p2 finishes at 7, p3 at 10, p1 at 30. waiting = 6, 4, 7 so avg wait = 5.67 ms.

Priority scheduling

every process has a priority number, lower number = higher priority (in our book). can be preemptive or not.
starvation of low priority procs. fix = aging: slowly raise the priority of procs that keep waiting (eg +1 every 15 min).

Multilevel queues

multilevel queue: separate queues, eg foreground (round robin) and background (FCFS). fixed priority between the queues.
multilevel feedback queue: a process can move between queues. one that uses too much cpu gets moved down. most general but also most complex to set up.

Metrics (exam q)

cpu utilization, throughput.
turnaround time = completion - arrival.
waiting time = time spent sitting in the ready queue.
response time = first response - arrival.

Deadlocks (OS lec 9)

deadlock = a set of procs where each one waits for a resource held by another one in the set, so nobody ever moves.
4 conditions must ALL hold: mutual exclusion, hold and wait, no preemption, circular wait. break any one and deadlock cant happen.
resource allocation graph: cycle + single instance per resource = deadlock for sure. with multiple instances a cycle only means maybe.
handling: prevention (break a condition), avoidance (banker's algorithm, needs max claims up front), detection + recovery, or just ignore it (ostrich, what unix does).
banker's algorithm: grant a request only if the system stays in a safe state, i.e. there is some order in which every proc can finish.
recovery = kill procs one by one or preempt resources (roll back a victim).

Semaphores and mutex

race condition = result depends on the order of interleaved execution. the critical section is the code that touches the shared data.
need mutual exclusion, progress and bounded waiting for a correct solution.
semaphore S = integer with two atomic ops: wait(S) decrements and blocks if it would go below 0, signal(S) increments and wakes one waiter.
binary semaphore (0/1) is like a mutex lock. counting semaphore counts available units of a resource.
classic problems: producer-consumer (bounded buffer), readers-writers, dining philosophers.
dining philosophers with 5 forks: if everyone grabs the left fork first -> deadlock.
fix = allow at most 4 to sit, or pick up both forks atomically.

Paging

logical address = page number + offset. page table maps page -> frame. page size is a power of 2 (eg 4 KB so offset = 12 bits).
internal fragmentation = unused space inside the last page of a process. paging has no external fragmentation.
TLB = small fast cache of page table entries. effective access time = hit ratio * (tlb + mem) + (1 - hit ratio) * (tlb + 2 * mem).
eg tlb = 10 ns, mem = 100 ns, hit ratio 80% -> EAT = 0.8 * 110 + 0.2 * 210 = 130 ns.

Page replacement

page fault = needed page is not in memory, OS has to bring it in from disk.
FIFO: replace the oldest page. suffers from Belady's anomaly (more frames can give MORE faults).
OPT: replace the page not needed for the longest time. optimal but needs the future, so only a yardstick.
LRU: replace the least recently used page. good in practice, no Belady's anomaly. needs hardware support (counter or stack).
thrashing = proc spends more time paging than executing. fix = working set model or reduce the degree of multiprogramming.